Restful Booker API — Defect & Bug Report

Local Instance (localhost:3001) | QA Take-Home Assessment Part 2

Author: Uma Uka **Target:** Docker Restful Booker Instance **Date:** August 30, 2026

Executive Overview

All defects below were found by the automated API test suite in `/api-tests` and independently reproduced via direct HTTP requests against a local Restful Booker instance (`docker run -d -p 3001:3001 mwinteringham/restfulbooker`).

Each defect is tracked in the automated suite via Playwright's `test.fail()`, which keeps assertions strict (encoding spec-correct behavior) rather than weakening them to force a pass. Bug IDs below are placeholders (BUG-1..6) pending formal tracker assignment.

Defect Summary Matrix

ID	Severity	Endpoint	Summary Description
BUG-1	LOW	`DELETE /booking/{id}`	Returns 201 Created instead of 200/204 on successful deletion.
BUG-2	LOW	`DELETE /booking/{id}`	Returns 405 Method Not Allowed instead of 404 when booking is missing/deleted.
BUG-3	HIGH	`POST /booking`	Missing required field causes 500 Internal Server Error instead of 400 Bad Request.
BUG-4	HIGH	`POST /booking`	Wrong-typed field is silently coerced to null instead of being rejected with 400.
BUG-5	MEDIUM	`POST /booking`	Inverted checkin/checkout date range is accepted without domain validation.
BUG-6	HIGH	`GET /booking`	Date-range filtering fails to match or return correct bookings under all test cases.

Detailed Defect Reports

BUG-1: DELETE returns 201 Created instead of 200/204 on success

LOW SEVERITY

Endpoint: DELETE /booking/{id}

Preconditions: A valid auth token (POST /auth) and an existing booking ID.

Steps to Reproduce:

1. POST /auth with valid admin credentials to obtain a token.
2. POST /booking with a valid payload to create a booking; note the returned `bookingid`.
3. Send DELETE /booking/{bookingid} with header `Cookie: token=<token>`.
4. Observe returned HTTP status code.

Request Sample:

```
DELETE /booking/52
Cookie: token=<token>
```

Expected Result: HTTP 200 OK or 204 No Content, per REST conventions for successful deletion.

Actual Result: HTTP 201 Created, with plain-text body "Created".

Impact Analysis: RFC 7231 §6.3.2 defines 201 as resource creation. Returning 201 for DELETE directly contradicts HTTP status semantics and misleads automated tooling checking status codes for deletion confirmation.

Automated Test Coverage: `api-tests/tests/crud.spec.js` — "DELETE returns a REST-correct success status" (tracked via `test.fail()`).

BUG-2: DELETE on non-existent booking returns 405 instead of 404

LOW SEVERITY

Endpoint: DELETE /booking/{id}

Preconditions: A valid auth token; a booking ID known not to exist (already deleted or never created).

Steps to Reproduce:

1. POST /auth to obtain a valid token.
2. POST /booking to create a booking, then DELETE it once successfully.
3. Send DELETE /booking/{same_id} again with the same valid token.
4. Observe returned HTTP status code (also reproducible against non-existent IDs like 99999999).

Request Sample:

```
DELETE /booking/54
Cookie: token=<token>
```

Expected Result: HTTP 404 Not Found.

Actual Result: HTTP 405 Method Not Allowed, with plain-text body "Method Not Allowed".

Impact Analysis: DELETE is a supported method on this route. Returning 405 signals that the method itself is disallowed on the route, breaking client-side idempotent retry logic that expects 404.

Automated Test Coverage: `api-tests/tests/crud.spec.js` — "deleting an already-deleted booking returns 404" (tracked via `test.fail()`).

BUG-3: Missing required field returns 500 Internal Server Error instead of 400

HIGH SEVERITY

Endpoint: POST /booking

Preconditions: None.

Steps to Reproduce:

1. Send POST /booking with a JSON body omitting the required `firstname` field.
2. Observe HTTP status code and response body.
3. Confirm via GET /booking that no new record was persisted.

Request Payload:

```
POST /booking
Content-Type: application/json

{
  "lastname": "NoFirst",
  "totalprice": 100,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-10-01",
    "checkout": "2026-10-05"
  }
}
```

Expected Result: HTTP 400 Bad Request with a validation payload identifying the missing field.

Actual Result: HTTP 500 Internal Server Error, with body "Internal Server Error".

Impact Analysis: Malformed client input should yield 4xx validation errors. Returning 500 indicates unhandled server exceptions, signaling poor application robustness and risking internal details leakage in production.

Automated Test Coverage: `api-tests/tests/negative-boundary.spec.js` — "rejects create with a missing required field" (tracked via `test.fail()`).

BUG-4: Wrong-typed field silently coerced to null instead of rejected

HIGH SEVERITY

Endpoint: POST /booking

Preconditions: None.

Steps to Reproduce:

1. Send POST /booking with `totalprice` passed as a string instead of a number.
2. Observe HTTP status code.
3. Inspect `totalprice` in the response payload.

Request Payload:

```
POST /booking
Content-Type: application/json

{
  "firstname": "Type",
  "lastname": "Test",
  "totalprice": "not-a-number",
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-11-01",
    "checkout": "2026-11-05"
  },
  "additionalneeds": "none"
}
```

Expected Result: HTTP 400 Bad Request, rejecting invalid data type.

Actual Result: HTTP 200 OK; booking is created but `totalprice` is silently stored and returned as `null`.

Impact Analysis: Silent data corruption. The client receives HTTP 200 without realizing data was dropped, leaving records incomplete.

Automated Test Coverage: `api-tests/tests/negative-boundary.spec.js` — "rejects create with a wrong-typed field" (tracked via `test.fail()`).

BUG-5: Accepts inverted date range (checkout before checkin)

MEDIUM SEVERITY

Endpoint: POST /booking

Preconditions: None.

Steps to Reproduce:

1. Send POST /booking with `bookingdates.checkin` later than `bookingdates.checkout`.
2. Observe HTTP status code and returned booking dates.

Request Payload:

```
POST /booking
Content-Type: application/json

{
  "firstname": "Date",
  "lastname": "Test",
  "totalprice": 100,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-11-10",
    "checkout": "2026-11-01"
  },
  "additionalneeds": "none"
}
```

Expected Result: HTTP 400 Bad Request, rejecting logically invalid date range.

Actual Result: HTTP 200 OK; booking created with inverted dates stored verbatim.

Impact Analysis: Domain validation gap. Downstream systems calculating length of stay, nightly pricing, or reporting will compute negative durations.

Automated Test Coverage: `api-tests/tests/negative-boundary.spec.js` — "rejects create where checkout is before checkin" (tracked via `test.fail()`).

BUG-6: GET /booking date-range filtering does not filter results correctly

HIGH SEVERITY

Endpoint: GET /booking?checkin={date}&checkout={date}

Preconditions: At least one known booking with a specific date range.

Steps to Reproduce:

1. Create booking with known date range (e.g., checkin=2026-12-01, checkout=2026-12-05); note ID.
2. Send GET /booking?checkin=2026-12-01 and observe.
3. Send GET /booking?checkout=2026-12-05 and observe.
4. Send GET /booking?checkin=2026-12-01&checkout=2026-12-05 and observe.
5. Send GET /booking?checkin=2020-01-01&checkout=2030-01-01 (wide range including all bookings) and observe.

Expected Result: Queries return exactly the bookings falling within range. Step 5 should trivially return all bookings.

Actual Result: Step 2 returns empty array `[]`. Step 3 ignores filter and returns all bookings. Step 4 & 5 return inconsistent subsets excluding matching bookings.

Impact Analysis: Core advertised search capability is completely non-functional under tested combinations.

Automated Test Coverage: `api-tests/tests/filtering.spec.js` — "GET /booking filtered by checkin/checkout date range returns exactly matching bookings" (tracked via `test.fail()`).

Appendix: Verified-Safe Behavior (Not a Defect)

SQL-Injection-Style Input Handled Safely in Text Fields

VERIFIED SAFE

Endpoint: POST /booking

Test Steps:

1. Send POST /booking with `firstname` set to `Robert'); DROP TABLE bookings;--`
2. Inspect `firstname` in response payload.
3. Send GET /booking/{id} and verify `firstname` value again.

Result: HTTP 200 OK. Value stored and returned verbatim with no execution, corruption, or server errors. System remained healthy (GET /ping).

Automated Test Coverage: `api-tests/tests/negative-boundary.spec.js` — "handles SQLi-style input safely" (expected to pass).